

CinéTalk — Script de soutenance

Projet B1 · ProjJS.2 Forum — Shakil Khaldi, Hugo Berton, Nohan Reis · 10 min de présentation + 5 min de questions

L'idée directrice : on raconte une **histoire** — celle d'un spectateur qui voudrait discuter d'un film et n'a nulle part où le faire. CinéTalk est la salle qu'on lui construit. On garde le fil rouge « cinéma » du début à la fin (scènes, projection, générique).

Rôles : **Shakil** pitch + équipe · **Hugo** fonctionnel + démo (clavier) · **Nohan** technique + conclusion.

Règle d'or : tout le monde parle, on ne lit pas les slides, on raconte. Respirez, regardez le jury.

Minutage (≈ 10 min)

Temps	Partie	Qui	Slides
0:00–1:00	Ouverture & pitch	Shakil	1–2
1:00–2:30	Fonctionnel + rôles	Hugo	3–4
2:30–3:15	Conformité 12/12	Shakil	5
3:15–6:00	Technique (stack, archi, C4, données, sécurité)	Nohan	6–10
6:00–7:00	Catalogue + cache	Hugo	11
7:00–9:00	DÉMO en direct	Hugo + tous	—
9:00–10:00	Équipe · difficultés · conclusion	tous	12–14

1 · Ouverture & pitch

0:00–1:00

Shakil

Slide 1 – titre

« Bonjour à tous. Imaginez : vous sortez d'un film, vous avez un avis, une théorie sur la fin... et personne avec qui en parler. Les avis sont éparpillés entre Reddit, les commentaires YouTube, les groupes. C'est de là qu'est né CinéTalk. »

Slide 2 – pitch

« CinéTalk, c'est un forum communautaire dédié au cinéma et aux séries. On s'est inspirés de Stack Overflow pour la structure en questions, de Reddit pour les votes, et de Discord pour l'esprit communautaire — mais entièrement tourné vers le 7^e art. Concrètement : on ouvre des fils de discussion, classés par genres, on échange des messages, on réagit, et on peut explorer un catalogue de films. C'est une application web complète, développée en Go avec une base MySQL. »

2 · Présentation fonctionnelle + rôles

1:00–2:30

Hugo

Slide 3 – fonctionnalités

« Voyons ce qu'on peut faire concrètement. On crée un compte et on se connecte, par pseudo ou par e-mail. Une fois membre, on ouvre des fils de discussion, on publie des messages, on les modifie ou on les supprime. On réagit aux messages avec des like et des dislike, qui donnent un score de popularité. On classe les fils par genres, on les trie, on les pagine, et on peut rechercher un sujet. Enfin, les administrateurs modèrent la plateforme. »

Slide 4 – cas d'utilisation

« On a d'abord modélisé tout ça. On a identifié trois rôles : le visiteur, qui peut seulement consulter ; le membre, connecté, qui peut publier, réagir et rechercher ; et l'administrateur, qui modère et peut bannir. Chaque rôle hérite des droits du précédent. Ce diagramme de cas d'utilisation a guidé tout notre développement. »

3 · Conformité au cahier des charges

2:30–3:15

Shakil

Slide 5 – 12/12

« Le sujet imposait douze fonctionnalités obligatoires, chacune avec ses règles de gestion précises. On les a toutes implémentées, et surtout vérifiées une par une : par exemple, un fil archivé devient totalement inaccessible, un message ne peut être posté que dans un fil ouvert, et un utilisateur ne peut avoir qu'une seule réaction active par message. Douze sur douze, conforme. »

4 · Présentation technique

3:15–6:00

Nohan

Slide 6 – la stack

« Côté technique. Le cœur de l'application est écrit en Go : il gère les routes, le rendu des pages, les formulaires, l'accès à la base et la sécurité. La base, c'est MySQL, relationnelle. Le JavaScript, lui, sert uniquement à l'expérience utilisateur — les réactions en temps réel sans recharger la page. Et tout se lance avec Docker, en une commande. »

Slide 7 – architecture en couches

« On a structuré le code en couches, sur le modèle MVC. Une requête traverse le Router, puis le Controller qui valide les entrées, le Service qui contient la logique métier, et le Repository qui parle à la base. Chaque couche a un seul rôle : c'est plus lisible, plus testable, et ça nous a permis de nous répartir le travail. »

Slide 8 – vue conteneurs (C4)

« Vue d'ensemble avec un diagramme C4 : le navigateur dialogue avec notre application Go, qui lit et écrit dans MySQL, et qui interroge l'API externe TMDB pour le catalogue. Quatre briques, des échanges clairs. »

Slide 9 — modèle de données

« La base compte six tables reliées par des clés étrangères : utilisateurs, fils, messages, réactions, tags, et une table de liaison. On a mis des suppressions en cascade — supprimer un fil supprime ses messages et ses réactions — et une contrainte d'unicité pour empêcher un double vote. »

Slide 10 — sécurité

« La sécurité, on l'a pensée dès le départ. Les mots de passe ne sont jamais stockés en clair : ils sont hachés en SHA-512 avec un sel unique. Les sessions reposent sur un jeton JWT, dans un cookie sécurisé, vérifié à chaque requête. Et toutes nos requêtes SQL sont paramétrées, pour éviter les injections. »

5 · Catalogue & performance

6:00–7:00

Hugo

Slide 11 — catalogue + cache

« En bonus, on a branché un vrai catalogue cinéma sur l'API TMDb : films, séries, acteurs. Mais une API externe, c'est lent et limité. Donc on a ajouté un cache en mémoire : la première consultation interroge TMDb, les suivantes sont servies depuis le cache pendant dix minutes. Résultat, une fiche passe de 0,35 seconde à 0,01 — vingt-six fois plus rapide. Et surtout, le catalogue n'est pas isolé : depuis chaque film, on peut ouvrir une discussion sur le forum. Je vous montre. »

6 · DÉMO EN DIRECT

7:00–9:00

Pilote : **Hugo** (clavier) · **Nohan** répond aux questions techniques · **Shakil** commente. **Avant de commencer :** avoir un onglet déjà connecté en secours. App sur **localhost:8080**. Comptes : **alice / Cinephile@2026** et **admin / Admin@CineTalk1**.

1. Accueil (non connecté) — montrer la liste des fils, les genres, la pagination 10/20/30/Tout.

« Voici l'accueil : les fils classés par genre, avec une pagination configurable. »

2. Tenter la recherche sans être connecté → redirige vers la connexion.

« La recherche est réservée aux membres : si je l'utilise sans être connecté, on me demande de m'identifier. »

3. Se connecter avec **alice**.

« Je me connecte en tant que membre. »

4. Catalogue → ouvrir un film → ★ **Ajouter aux favoris**, écrire un **commentaire**, puis montrer « **Mes favoris** » dans le menu.

« Le catalogue n'est pas qu'une vitrine : sur une fiche film, je peux l'ajouter à mes favoris — le compteur de membres se met à jour — et laisser un commentaire, comme une mini-communauté autour du film. Et je retrouve tout dans « Mes favoris ». »

5. Sur la fiche film, cliquer « **En discuter sur le forum** ».

« Et si je veux un vrai débat, le bouton « En discuter sur le forum » ouvre un sujet déjà pré-rempli avec le film : le catalogue et le forum sont reliés. »

6. Choisir un genre, publier le fil → poster un message → liker/disliker (montrer la mise à jour sans rechargement) → changer le tri (récent / populaire).

« J'ajoute un genre, je publie. Je poste un message, je réagis — le score se met à jour instantanément, sans recharger la page, c'est le rôle du JavaScript. Et je peux trier par popularité. »

7. Ouvrir un fil **fermé** (lecture seule), puis tenter un fil **archivé** par URL → page introuvable.

« Un fil fermé reste consultable mais n'accepte plus de messages. Et un fil archivé est totalement inaccessible, même en tapant son adresse directement. »

8. Se déconnecter, se reconnecter en **admin** → tableau de bord → changer l'état d'un fil, supprimer un message d'un autre, bannir un compte.

« En administrateur, j'accède au tableau de bord : je peux changer l'état d'un fil, supprimer n'importe quel message, et bannir un utilisateur — qui perd alors immédiatement l'accès. »

7 · Coulisses, difficultés & conclusion

9:00–10:00

Shakil

Slide 12 – équipe

« On était trois. Je me suis occupé de l'authentification et de l'administration, Hugo des fils, de la recherche, du catalogue et de l'interface, et Nohan des messages, des réactions et de la base. On a suivi nos tâches sur Trello, une carte par fonctionnalité, et travaillé ensemble sur GitHub. »

Hugo

Slide 13 – difficultés

« La plus grosse difficulté, c'était de penser en couches sans tout mélanger — on a appris à respecter les responsabilités de chacune. L'authentification par JWT et l'intégration de l'API TMDb nous ont aussi demandé de la réflexion, d'où le cache. »

Nohan

Slide 14 – conclusion

« Pour conclure : un projet complet, conforme à 100 % au cahier des charges, sécurisé, et installable en une commande. Si on continuait, on ajouterait des notifications en temps réel, des profils utilisateurs, et on lierait encore plus le catalogue au forum. Ce projet nous a fait mobiliser toutes les notions de l'année. Merci, on répond à vos questions. »

Préparation aux questions (5 min)

Pourquoi du rendu serveur et pas une vraie API + front JavaScript ?

Choix assumé : le sujet impose Go pour le métier et le JS seulement pour l'UX. Le rendu serveur est plus simple, plus rapide à charger et plus sûr (pas de logique sensible côté client). Le JS est utilisé là où il apporte vraiment quelque chose : les réactions asynchrones.

Le catalogue : vous rappelez l'API à chaque fois ?

Non. On a un cache mémoire avec une expiration de 10 minutes : premier appel sur l'API, les suivants depuis le cache, 26× plus rapide. Le TTL garantit des données fraîches.

Comment empêchez-vous un double like sur un message ?

Par une contrainte d'unicité en base sur (message, utilisateur), plus une logique de bascule côté service : on ne peut avoir qu'une seule réaction active, et liker puis disliker remplace l'ancienne.

Comment sont protégés les mots de passe ?

Hachage SHA-512 avec un sel unique par compte, jamais en clair. Règles imposées : 12 caractères minimum, une majuscule, un caractère spécial.

Un utilisateur peut-il supprimer le message d'un autre ?

Non, sauf l'administrateur. On vérifie côté service que l'utilisateur est soit le propriétaire, soit admin, avant toute modification ou suppression.

Que se passe-t-il quand on supprime un fil ?

Les données liées sont supprimées en cascade : ses messages, et les réactions de ces messages. C'est géré par les clés étrangères ON DELETE CASCADE en base.

Comment la recherche sait si on cherche un titre ou un genre ?

Le système détermine automatiquement : il compare la saisie aux titres et aux genres existants et renvoie les résultats cohérents, en excluant les fils archivés.

Pourquoi Docker ?

Pour que le projet soit installable et testable en une commande, avec le même environnement pour les trois membres et pour le correcteur : l'app et MySQL démarrent ensemble, et la base est initialisée automatiquement.